



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Food Rescue Connect: Smart Food Donation and Redistribution Management System Industry Context

A CASE STUDY REPORT

Submitted in the partial fulfilment for the Course of

CSA1016-Software Engineering

to the award of the degree of
BACHELOR OF ENGINEERING
IN
CSE-AI

Submitted by
Sk. Ashwaq Ahamed (192572070)

Under the Supervision of
Dr. K. ANITA DAVAMANI

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

1. Problem Analysis and Industry Context

Food waste is a significant global challenge occurring alongside continuing food insecurity. Restaurants, supermarkets, hotels, event organizers and food manufacturers may dispose of surplus edible food because available donation and distribution processes are not sufficiently coordinated. Governments and non-governmental organizations promote food recovery and sustainable distribution initiatives, creating a need for digital platforms that can connect food sources with organizations and communities that need them. FoodRescue Connect is proposed as a smart food donation and redistribution management system. The platform centralizes information about food availability, quantity, expiry time, donor location, pickup requests and delivery status. It is designed to reduce delays between donation and collection and to improve coordination between donors, NGOs, volunteers and beneficiaries.

Problem Statement

Large quantities of safe and nutritious food may be discarded because donors and charitable organizations lack efficient mechanisms for coordinating collection and distribution. Manual communication can result in delayed pickups, spoilage and uneven allocation. Beneficiary organizations may also have difficulty identifying nearby food sources and managing distribution efficiently. The proposed system addresses these issues through centralized data management, real-time status tracking, notifications and smart prioritization.

Proposed Solution

- Provide a centralized platform for donors, NGOs, volunteers and beneficiaries.
- Allow donors to register surplus food with quantity, location, quality information and expiry time.
- Allow NGOs to discover and accept available donations.
- Allow volunteers to manage pickup and delivery tasks.
- Prioritize donations that are closer to expiry so urgent food can be handled first.
- Provide dashboards and reports for donation, collection and distribution activities.

Expected Value

The intended value is faster food collection, better allocation, improved transparency and reduced avoidable food waste. The system also creates a technical foundation that can later be extended with GPS, IoT sensors, advanced AI models and mobile applications.

2. Objectives, Requirements and Expected Outcomes

Project Objectives

1. Develop an intelligent platform for food donation and redistribution management.
2. Connect restaurants, supermarkets, NGOs, volunteers and beneficiaries through a centralized system.
3. Monitor food availability, quantity, expiry time and quality.
4. Apply smart algorithms to support food allocation and transportation decisions.
5. Generate notifications for available donations and pickup requests.
6. Track food collection, delivery status and beneficiary distribution.
7. Produce analytical reports on donation activity and food waste reduction.
8. Promote sustainable food recovery and community welfare through digital technology.

Functional Requirements

ID	Requirement	Description
FR1	User Management	Register/login and maintain role-based user information.
FR2	Donation Management	Create, edit, view and track food donations.
FR3	Food Details	Store food name, quantity, location, expiry time and status.
FR4	NGO Management	View available donations and accept suitable requests.
FR5	Volunteer Management	Accept pickup tasks and update collection/delivery status.
FR6	Priority Handling	Assign urgency based on remaining shelf life.
FR7	Notifications	Inform relevant users about donations and pickup requests.
FR8	Dashboard	Display donation, pickup and delivery statistics.
FR9	Tracking	Maintain status from donation through final delivery.

Non-Functional Requirements

- Usability – interfaces should be simple for donors, NGOs and volunteers.
- Performance – common operations should return quickly for normal loads.
- Security – user authentication and protected application endpoints should be used.
- Scalability – the design should allow additional users, donors and locations.
- Maintainability – modular frontend/backend code and Git history should support changes. □ Portability – Docker should allow the application to run consistently across environments.

Expected Outcomes

- Reduction in avoidable edible food waste.
- Faster and more efficient food collection and redistribution.
- Improved access to available food for beneficiary communities.
- Better coordination among donors, NGOs and volunteers.
- Transparent tracking of donation and delivery status.
- A scalable foundation for sustainable digital food recovery.

3. Software Architecture and Component Design

A layered web architecture is selected because FoodRescue Connect contains a user interface, application logic, APIs and persistent data. The architecture separates responsibilities so that individual components can be developed and maintained independently. The design also fits the Git and Docker requirements because the frontend, backend and database can be managed as separate services.

FoodRescue Connect - High-Level Architecture

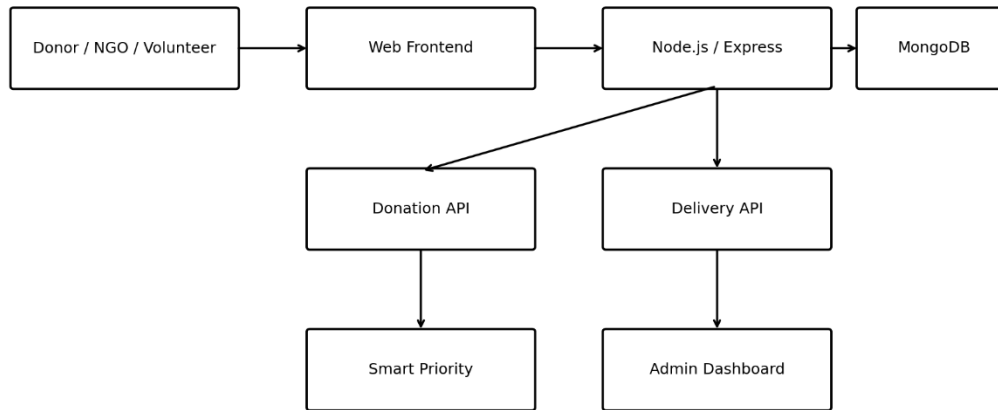


Figure 1. High-level architecture of FoodRescue Connect

Architecture Components

Component	Responsibility
Frontend	Provides pages for login, donation entry, dashboards, NGO actions and volunteer tasks.
Backend API	Processes requests, validates data and connects the frontend with the database.
Donation API	Creates, retrieves and updates donation records.
Delivery API	Manages pickup and delivery status.
Smart Priority Module	Assigns urgency using remaining expiry time.
MongoDB	Stores users, donations and delivery information.
Admin Dashboard	Provides management and activity summaries.

Component Interaction

A donor submits a donation through the frontend. The backend validates the information and stores it in MongoDB. The smart priority component evaluates the remaining expiry period and assigns an urgency level.

NGOs can view suitable donations and accept them. Volunteers then receive pickup information and update the delivery status. The dashboard uses stored records to display current activity and summary information.

Quality Attributes

- Modularity: separate frontend, backend, database and service responsibilities.
- Portability: Docker packages the application environment.
- Maintainability: Git branches and meaningful commits provide controlled evolution.
- Scalability: the API-based design can later be expanded to cloud infrastructure.

4. Project Structure, Data Design and Smart Logic

Repository Folder Structure

FoodRescue-Connect/

```
├──frontend/
|   ├──index.html
|   ├──login.html
|   ├──dashboard.html
|   ├──donation.html
|   ├──css/style.css
|   └──js/app.js
├──backend/
|   ├──server.js
|   ├──package.json
|   ├──routes/
|   ├──models/
|   └──controllers/
├──docker/
|   ├──Dockerfile
|   ├──docker-compose.yml
|   └──.gitignore
└── README.md
```

The frontend folder contains user-facing pages and browser scripts. The backend folder contains the Express server, API routes, data models and controllers. Docker configuration is kept at repository level so the complete application can be built and started consistently. README.md documents setup and execution instructions, while .gitignore prevents unnecessary local files from entering version control.

Core Data Entities

Entity	Important Fields
User	user_id, name, email, password, role, location
Donation	donation_id, food_name, quantity, expiry_time, location, donor_id, status, priority
Delivery	delivery_id, donation_id, volunteer_id, ngo_id, pickup_location, delivery_location, status

Donation Status Lifecycle

A donation can move through the following states: Available → Accepted → Picked Up → Delivered. If the food is no longer suitable or reaches its expiry condition, the status can be changed to Expired. Maintaining a clear lifecycle makes the system easier to monitor and report.

Simple Smart Priority Logic

If remaining expiry time < 2 hours:

Priority = HIGH Else if remaining

expiry time <= 5 hours:

Priority = MEDIUM

Else:

Priority = LOW

This rule is intentionally simple for the technical assignment. It demonstrates how the system can prioritize urgent donations without requiring a complex machine-learning model. A future version can replace the rule with a trained prediction model using historical donation and demand data.

5. Git Repository, Branching Strategy and Version Control Workflow

Git is used to manage source-code versions and demonstrate collaborative development. The assignment requires repository initialization, a suitable branching strategy, meaningful commits, branch creation, merging, synchronization and commit-history analysis.

Repository Initialization

```
mkdir FoodRescue-  
Connect cd  
FoodRescue-Connect  
git init git add . git commit -m  
"Initial project structure" git branch  
-M main
```

Branching Strategy

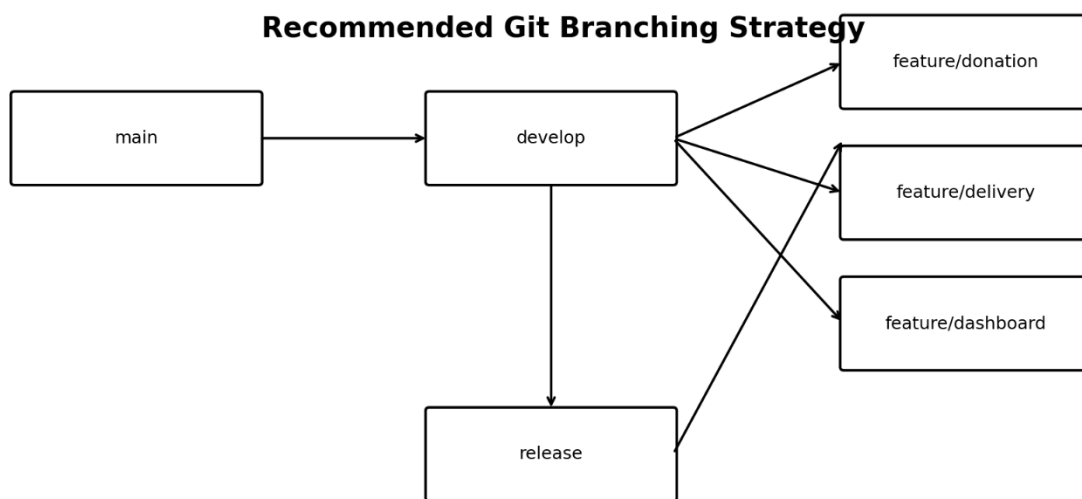


Figure 2. Recommended Git branching strategy

The main branch represents stable code. The develop branch integrates completed work before release. Feature branches isolate individual changes such as donation, delivery and dashboard functionality. A release branch can be used to prepare a submission version. This approach reduces the risk of unstable changes being placed directly into main.

Typical Workflow

```
git checkout develop
git checkout -b feature/donation
git add .
git commit -m "Add food donation module"
git checkout develop
git merge feature/donation
git checkout main
git merge develop
git push -u origin main
```

Recommended Commit History

- Initial project structure
- Add frontend homepage
- Add donor registration
- Add food donation form
- Implement donation API
- Add MongoDB integration
- Add NGO dashboard
- Add volunteer pickup module
- Add donation priority logic
- Add Dockerfile
- Add Docker Compose configuration
- Update README documentation

Commit messages are written as short action-oriented descriptions. This makes the project history easier to understand and helps identify which change introduced a feature or fix.

6. Docker Environment Design and Implementation

Docker is suitable for FoodRescue Connect because the application contains multiple components with different runtime requirements. Containerization packages the application and its dependencies so the project

can be built and executed consistently on another machine. Docker Compose can coordinate the frontend, backend and database services.

Container Design

Service	Purpose	Example Port
Frontend	Serves the web interface.	3000
Backend	Runs Node.js/Express APIs.	5000
MongoDB	Stores application data.	27017

Dockerfile

```
FROM node:20
WORKDIR
/app
COPY package*.json
./
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node",
"server.js"]
```

FROM selects the Node.js base image. WORKDIR sets the application directory. COPY transfers package information and source files. RUN npm install installs dependencies. EXPOSE documents the backend port. CMD starts the server when the container launches.

Docker Compose Configuration

```
services:
  backend:
    build:
      ./backend
    ports:
      -
        "5000:5000"
    depends_on:
      -
        mongodb
```

```

frontend:
  build:
    ./frontend
  ports:
    - "3000:3000"
  depends_on:
    - backend

mongodb:
  image:
    mongo:latest
  ports:
    - "27017:27017"

```

The Compose file defines the services and their dependencies. In a complete implementation, named volumes can be added for persistent MongoDB storage and environment variables can be used for database connection settings. A shared Docker network allows services to communicate by service name.

Build and Run Commands

```

docker compose
build      docker
compose    up
docker ps  docker
compose down

```

7. Container Deployment, Testing and Evidence

The deployment test verifies that the application can be started using Docker and that the main user flow operates correctly. The assignment requires evidence that the application runs successfully, so screenshots should be inserted in this section after the practical execution.

Deployment Procedure

Functional Test Cases

Test ID	Test Case	Expected Result
TC01	Open application	Homepage loads successfully.
TC02	User login	Valid user reaches the appropriate dashboard.
TC03	Create donation	Donation is stored with correct details.
TC04	Priority calculation	Donation receives an urgency level.
TC05	NGO accepts donation	Status changes from Available to Accepted.
TC06	Volunteer pickup	Status changes to Picked Up.
TC07	Complete delivery	Status changes to Delivered.
TC08	Dashboard	Donation and delivery statistics are displayed.

8. Benefits, Component Interaction and Challenges

Benefits of Git

- Maintains a complete history of project changes.
- Allows feature development in isolated branches.
- Supports merging and collaboration.
- Makes it easier to identify and revert problematic changes.
- Provides evidence of development activity for the assignment.

Benefits of Docker

- Provides a consistent execution environment.
- Packages application dependencies with the service.
- Separates frontend, backend and database responsibilities.
- Simplifies deployment and setup on another machine.

- Supports scalable multi-service deployment through Docker Compose.

Component Interaction and Quality Analysis

Quality Attribute	FoodRescue Connect Approach
Maintainability	Modular frontend/backend structure and Git history.
Portability	Docker containers package runtime dependencies.
Scalability	API-oriented backend can be expanded to additional services.
Usability	Role-based dashboards simplify donor, NGO and volunteer tasks.
Reliability	Donation status lifecycle provides clear tracking.
Performance	Simple APIs and indexed data can support normal application workloads.
Security	Authentication, role checks and protected API endpoints should be implemented.

Challenges

- Keeping donation status synchronized between multiple user roles.
- Handling food expiry and ensuring urgent donations receive attention.
- Managing database connectivity between containers.
- Resolving Git merge conflicts when multiple branches modify the same files.
- Configuring Docker networking and service dependencies.
- Providing real-time notifications requires additional infrastructure in a production system.

Mitigation Approach

The project uses a controlled status lifecycle, modular API endpoints, feature branches and Compose service dependencies. Testing is performed after each major module and again after containerization. Future versions can introduce automated tests and continuous integration to reduce deployment errors.

9. Future Enhancements, Conclusion and Submission Checklist

Future Enhancements

17. GPS and map integration for accurate donor, NGO and volunteer locations.
18. AI-based food demand prediction using historical donation and beneficiary data.
19. Advanced route optimization to reduce travel time and transportation cost.
20. IoT sensors for temperature and quality monitoring of sensitive food.
21. Mobile applications for volunteers and NGOs.
22. SMS, email and push notifications for urgent donations.
23. Cloud deployment with automated scaling.
24. Role-based security and stronger authentication.
25. Automated testing and CI/CD pipelines connected to the Git repository.
26. Analytics dashboards for food waste, carbon reduction and community impact.

Conclusion

FoodRescue Connect provides a structured digital approach to coordinating surplus food donations and redistribution. The proposed platform connects donors, NGOs and volunteers through a centralized application while maintaining information about food quantity, expiry, location and delivery status. The smart priority rule demonstrates how computational logic can help identify urgent donations and support timely action.

The Git workflow provides controlled version management, while Docker and Docker Compose provide a repeatable environment for deployment. Together, these technologies address the technical requirements of the assignment and create a foundation for future enhancements such as AI prediction, GPS route optimization, IoT monitoring and cloud deployment.

Final Submission Checklist

Deliverable	Status / Evidence
10-page technical report	Prepared – this document
Git repository	Create and submit GitHub repository URL
Complete commit history	Include meaningful commits and branch/merge history
Source code	Include frontend, backend and configuration files
Dockerfile	Include in repository

docker-compose.yml	Include in repository
Git screenshots	Insert actual screenshots from your execution
Docker screenshots	Insert actual build/run/container screenshots
Architecture diagram	Figure 1 included
Workflow diagram	See Figure 3 below / insert into report
Git branching diagram	Figure 2 included

FoodRescue Connect - Donation and Delivery Workflow

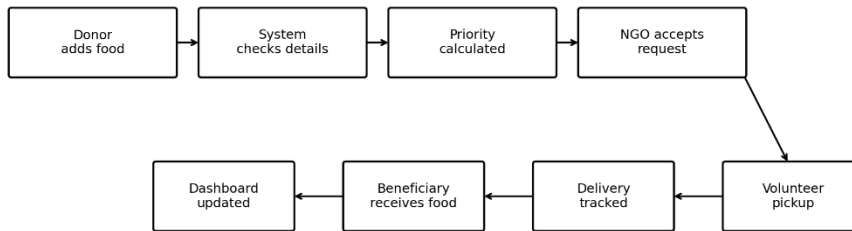


Figure 3. FoodRescue Connect donation and delivery workflow

References

- Course assignment brief: CO3 Assessment Tool 1 – Git & Docker Technical Assignment, Software Engineering, SIMATS Engineering.
- Project problem statement and objectives supplied for FoodRescue Connect.
- Git and Docker command examples are presented as implementation guidance for the assigned project.